# Ders 6 Genişletilmiş Notu: Öğrenci Sunumları: Konteynerle Dağıtım, Mikroservislere Yeniden Yapılandırma ve Bileşen Testleri

## Ders 6 Genişletilmiş Notu: Öğrenci Sunumları: Konteynerle Dağıtım, Mikroservislere Yeniden Yapılandırma ve Bileşen Testleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Servisleri konteyner olarak dağıtmak: Service as a Container deseni
- 2. Sanal makine ve konteyner karşılaştırması
- 3. Docker ve ana bileşenleri
- 4. Konteyner bazlı dağıtım süreci: build ve runtime
- 5. Docker Compose, orkestrasyon ve desenin değerlendirmesi
- 6. Neden bir monoliti yeniden yapılandıralım?
- 7. Monoliti boğmak: Strangler deseni ve stratejileri
- 8. Monolitik mimari ve geçişin gerekçeleri
- 9. Üç geçiş stratejisi
- 10. Servis çıkarmanın zorlukları: domain modeli ve veritabanı
- 11. Servis ile monolitin iletişimi: integration glue
- 12. Anti-corruption Layer deseni
- 13. Saga'larla veri tutarlılığı ve kimlik doğrulama
- 14. Bileşen testleri: stub, mock ve Gherkin
- 15. Bileşen testlerinin tasarımı, yazılması ve çalıştırılması


Bu ders beş öğrenci sunumundan oluşur. Hoca dersi önceki haftadan kalan sunumlarla başlatmak istedi, ancak o sunum dersin sonuna alındı. Sırasıyla şu sunumlar yapıldı: servisleri konteyner olarak dağıtmak (Bölüm 1-5), monoliti mikroservislere yeniden yapılandırmanın nedenleri ve Strangler deseni (Bölüm 6-7), monolitten mikroservislere geçiş stratejileri (Bölüm 8-10), servis ile monolitin iş birliğinin tasarımı (Bölüm 11-13) ve önceki haftadan kalan bileşen testleri (Bölüm 14-15). Planlanan iki sunum bu hafta da yapılamadı. Sunan kişiler bu notta "bir öğrenci" diye anılır. Hoca bu derste sunumları yönetti, akademik bir ek açıklama yapmadı. Dersin sonunda mikroservis mimarisinin ana hatlarıyla, değişik yönleri ve desenleriyle incelenmesinin bu dersle tamamlandığını söyledi.

> **Ek bilgi (derste anlatılmadı):** Sunumlardaki şekiller ve kod listeleri (FTGO, Listing 12.1, Figure 13.8, 13.9, 13.11 vb.) Chris Richardson'ın *Microservices Patterns* kitabından alınmıştır. Bu kitapta servis dağıtımı büyük olasılıkla on ikinci bölümde ("Deploying microservices"), mikroservislere yeniden yapılandırma on üçüncü bölümde ("Refactoring to microservices"), bileşen testleri ise onuncu bölümde yer alır.

### 1. Servisleri konteyner olarak dağıtmak: Service as a Container deseni

Öğrenci, dönem boyunca sunumlarda bir mikroservis mimarisinde servis oluşturma ve geliştirme konularının konuşulduğunu, geliştirme bittikten sonra ise servislerin dağıtılması (dep-

loyment) işinin geldiğini söyleyerek başlar.

### 1.1 Geleneksel yöntemlerden konteynere

Geleneksel yaklaşımda servisler fiziksel sunucuların üzerine ya da bir sunucu üzerinde oluşturulan sanal makinelere kurulurdu. Mikroservis mimarisinin yaygınlaşmasıyla çok sayıda servis ortaya çıktı. Her servisin farklı bağımlılıkları, farklı istekleri ve farklı ortam ihtiyaçları vardır. Bu da dağıtım sürecinde daha taşınabilir ve daha izole çözümlere ihtiyaç doğurdu. Bunun sonucunda Service as a Container deseni ortaya çıktı: her servis birbirinden bağımsız bir konteyner olarak oluşturulur, ayağa kaldırılır ve dağıtılır. Öğrenciye göre bu desen bugün modern mikroservis mimarilerinin temel yapı taşlarından biridir.

Öğrenci iki kavramı ayırır. Servis, bir uygulamanın belirli işlerini yerine getiren bağımsız yazılım bileşenidir. Konteyner ise uygulamayı, uygulamanın bağımlılıklarını ve çalışma ortamını içeren, kendi başına çalışan bir birimdir.

### 1.2 Desenin mikroservis mimarisindeki rolü

Her mikroservis bağımsız geliştirilir ve bağımsız çalıştırılır. Bunun sürdürülebilmesi için servislerin bağımsız dağıtılabilmesi ve birbirine bağımlı olmaması gerekir. Desenin katkıları şöyle özetlenir:

- Ölçeklenebilirlik: büyük bir mimaride ihtiyaç olduğunda hızla yeni bir servis ayağa kaldırmak, ihtiyaç kalmayınca kapatmak gerekir. Konteyner bunu kolaylaştırır.
- Teknoloji bağımsızlığı: mikroservis mimarisinin amaçlarından biri farklı dil ve teknolojileri bir arada kullanabilmektir. Ancak farklı dillerde yazılmış uygulamaların işletim sistemi düzeyinde farklı ihtiyaçları olabilir ve bunları aynı ortamda çalıştırmak sorun çıkarabilir. Servis konteyner olarak dağıtıldığında her servisin ihtiyaçları kendi konteynerinde kapsüllenir, bu sorun çözülür.

## 2. Sanal makine ve konteyner karşılaştırması

Slayttaki karşılaştırma iki yaklaşımı yan yana verir (ekrandan, 00:06:45).

Sanal makine (VM) yapısında katmanlar alttan üste şöyledir:

```
Infrastructure
  -> Hypervisor (VM Emulator)
     -> Virtual Machine 1, 2, 3 (her birinde):
          Guest Operating System
          Bins/Libraries
          Application
```

Konteyner yapısında ise:

```
Infrastructure
  -> Operating System
     -> Container Engine
        -> Container 1, 2, 3 (her birinde):
             Bins/Libraries
             Application
```

Öğrencinin anlatımıyla VM yaklaşımında fiziksel sunucunun üzerinde bir işletim sistemi kuruludur. Hypervisor denen bir program yardımıyla bunun üzerine farklı işletim sistemlerinde sanal makineler kaldırılır. Ama her sanal makinenin kendi işletim sistemi vardır. Bunların kurulması ve ayağa kalkması maliyetlidir, fiziksel sunucuda ek yer kaplar ve sunucuyu zorlar. Öğrenciye göre en basit bir işletim sistemi bile bir makineye konduğunda 20-30 GB yer kaplar. Konteyner yaklaşımında ise yeni bir işletim sistemi kurulmaz. Host işletim sisteminin üzerindeki konteyner motoru (container engine), birbirinden bağımsız küçük konteynerler çalıştırır. Konteynerler ortak imaj katmanlarını kullandığı için bir fiziksel sunucuda binlerce konteyner rahatça kaldırılabilir.

Slayttaki maddeler:

| Geleneksel dağıtım yöntemleri | Konteyner bazlı yöntemler |
|---|---|
| Her bir uygulama için bağımsız bir işletim sistemi çalıştırır. | Tek bir işletim sistemi çekirdeğini paylaşır. |
| VM'lerin başlama süresi oldukça uzundur. | Hızlı başlar ve düşük kaynak tüketimi sağlar. |
| Yüksek kaynak tüketimine neden olur. | Özellikle mikroservislerin küçük ve bağımsız yapısıyla uyumludur. |

## 3. Docker ve ana bileşenleri

### 3.1 Konteynerler neden önemli?

Slayta göre (ekrandan, 00:10:00) konteyner teknolojileri yazılım dağıtım süreçlerini dönüştürmüştür. Geliştiriciler konteynerler sayesinde kodlarını hızlıca test edebilir ve tutarlı bir şekilde taşıyabilir; bu da DevOps süreçlerini hızlandırırken maliyetleri düşürür. En yaygın konteyner teknolojileri olarak Docker ve Solaris verilir.

Öğrenci tutarlılığı yazılımcıların bilinen "benim makinemde çalışıyordu" sözüyle açıklar. Bir uygulama, geliştiricinin makinesinde bulunan bir işletim sistemi bağımlılığını kullanıyorsa, başka bir makineye taşındığında o bağımlılık orada olmadığı için çalışmayabilir. Konteyner, uygulamanın ihtiyaç duyduğu her şeyin kendi içinde olmasını garanti ettiği için bu sorunu ortadan kaldırır; test makinesi ile üretim makinesi aynı sonucu verir.

### 3.2 Docker nedir?

Docker, bir uygulamayı ve bütün bağımlılıklarını konteyner denen tek bir standart birime paketlemeyi sağlayan açık kaynaklı bir konteyner platformudur. Öğrenci Docker'ın sağladıklarını şöyle sıralar:

- Taşınabilirlik: oluşturulan konteyner geliştiricinin makinesinden test ortamına rahatça taşınır.
- Tekrarlanabilirlik: uygulamanın test ve üretim ortamında aynı şekilde ve aynı verimle çalışacağının garantisi.
- Verimlilik: bir fiziksel makineye birden çok işletim sistemi kurmaya gerek kalmaz. Bir makinede onlarca servis kaldırılacaksa onlarca işletim sistemi kurmak mümkün değildir; konteyner yapısı bu konuda verim sağlar.
- Ölçeklenebilirlik: servisler Docker yardımıyla çok hızlı ayağa kaldırılıp indirilebilir.

### 3.3 Docker'ın ana bileşenleri

Slayttaki diyagram ve maddeler şöyledir (ekrandan, 00:12:20):

```
[Container Engine]
  Dockerfile --Build--> Images --Run--> Container
                          |
                        Push
                          v
                    Registry/Hub  (birçok statik imajı saklar)
                          |
                        Pull
                          v
[Container Engine, başka makine]
  Images --Run--> Container
```

- Docker Engine: Docker'ın çekirdek parçasıdır; konteynerlerin oluşturulması ve yönetimiyle ilgilenir.
- Docker Image: konteyner oluşturmak için kullanılan, uygulama kodunu ve bağımlılıklarını içeren salt okunur bir şablondur. Konteynerler önceden oluşturulmuş bir imajdan kaldırılır. Öğrenciye göre imaj çok katmanlı (multi-layer) bir yapıdadır ve katmanlar farklı imajlar arasında ortak kullanılır; bu da ek verimlilik sağlar.

- Docker Hub/Registry: konteyner imajlarını bulmak ve paylaşmak için kullanılan bulut tabanlı depodur. Öğrenci bunu Java'daki Maven repository'ye ya da Node.js'teki npm'e benzetir: oluşturulan imajlar ortak bir yerde toplanır, farklı makineler ve farklı kişiler onları rahatça kullanır.
- Dockerfile: bir Docker imajı oluşturmak için talimatlar içeren dosyadır.

## 4. Konteyner bazlı dağıtım süreci: build ve runtime

Öğrenci konteyner bazlı dağıtımı iki aşamaya ayırır.

- Build aşaması: bir Dockerfile oluşturulur, imajı oluşturacak komutlar bu dosyaya tek tek yazılır. Servis ve bağımlılıkları oluşturulacak imajın içine gömülür. `docker build` komutuyla Dockerfile'dan bir Docker imajı elde edilir.
- Runtime aşaması: oluşturulan imaj bir registry'ye konur. Öğrenci örnek olarak Docker Hub'ı ve Amazon Web Services'teki Elastic Container Registry'yi verir. Sonra imaja ihtiyaç duyan makine onu registry'den çeker (pull). Kubernetes gibi bir konteyner orkestrasyon aracına "şu makinede şu servisi çalıştır" demek yeterlidir; araç ilgili imajı registry'den çekip o makinede ayağa kaldırır.

### 4.1 Dockerfile örneği

Öğrenci kitaptaki örneği paylaşır. Slayttaki Listing 12.1, Restaurant Service'i oluşturmak için kullanılan Dockerfile'dır (ekrandan, 00:16:20):

```
FROM openjdk:8u171-jre-alpine
RUN apk --no-cache add curl
CMD java ${JAVA_OPTS} -jar ftgo-restaurant-service.jar
HEALTHCHECK --start-period=30s --
    interval=5s CMD curl http://localhost:8080/actuator/health || exit 1
COPY build/libs/ftgo-restaurant-service.jar .
```

Slayttaki açıklamalara ve öğrencinin anlatımına göre satırlar şunu yapar:

- `FROM`: temel imaj. `openjdk:8u171-jre-alpine`, JRE'yi içeren, az yer kaplayan bir Linux imajıdır.
- `RUN apk --no-cache add curl`: sağlık kontrolünde kullanılmak üzere curl kurulur.
- `CMD`: konteyner başladığında `java -jar` ile servis JAR'ının çalıştırılmasını ayarlar.
- `HEALTHCHECK`: Docker'ı sağlık kontrolü uç noktasını çağıracak şekilde yapılandırır. `/actuator/health` uç noktası ilk 30 saniyelik gecikmeden sonra her 5 saniyede bir çağrılır.
- `COPY`: Gradle'ın build dizinindeki JAR'ı imajın içine kopyalar.

Slayttaki listede `HEALTHCHECK` satırı sayfa düzeni nedeniyle iki satıra bölünmüştür; `--start-period=30s` ve `--interval=5s` aynı yönergenin iki seçeneğidir.

Bu Dockerfile `docker build` ile derlendiğinde bir Restaurant Service imajı elde edilir.

### 4.2 İmajı registry'ye gönderme

Oluşturulan imajın farklı makinelerden erişilebilir olması için bütün makinelerin görebildiği ortak bir registry'ye gönderilir. Slayttaki komutlar şöyledir (ekrandan, 00:17:45; registry adresi burada `<registry>` diye kısaltıldı):

```
docker tag ftgo-restaurant-service <registry>/ftgo-restaurant-service:1.0.0.RELEASE
docker push <registry>/ftgo-restaurant-service:1.0.0.RELEASE
```

- `docker tag` versiyonlama ve isimlendirme için kullanılır. Öğrenci, imajın registry'ye gönderilmeden önce 1.0, 1.1, 1.2 gibi sürümlerle etiketlendiğini söyler.
- `docker push`, etiketlenmiş imajı verilen adrese gönderir.

Son adımda, registry'ye erişimi olan başka bir makinede `docker run` komutu çalıştırılır. İlgili imaj registry'den çekilir ve Dockerfile'da belirtilen `java -jar` komutu çalışır.

## 5. Docker Compose, orkestrasyon ve desenin değerlendirmesi

### 5.1 Docker Compose ve orkestrasyon

Yukarıdaki örnek tek bir servistir; mikroservislerde ise onlarca servis yönetilir. Docker'ın bunun için sunduğu Docker Compose, bir YAML dosyasıdır. Bu dosya ile birden çok konteyner tek bir komutla ayağa kaldırılıp indirilebilir ve servisler birbiriyle uyumlu biçimde başlar; çünkü bir servis başka bir servise ihtiyaç duyabilir. Ancak Docker Compose dosyası o makineye özgü bir yapılandırmadır. Geliştirme aşamasında yazılımcının işini kolaylaştırsa da büyük ölçekte, üretim ortamında yetersiz kalır. Farklı makinelerde farklı servisleri kaldırmak için orkestrasyona ihtiyaç vardır; en bilinen orkestrasyon aracı Kubernetes'tir, Docker'ın kendi aracı ise Docker Swarm'dır.

### 5.2 Değerlendirme

Slayttaki değerlendirme şöyledir (ekrandan, 00:20:00):

| Avantajlar | Dezavantajlar |
|---|---|
| İzolasyon | Sistem yönetimi yükü |
| Kaynak kullanımı | Orkestrasyon ihtiyacı |
| Hızlı dağıtım | Bağımlılıklar arası karmaşıklık |
| Taşınabilirlik | Güvenlik riskleri |
| Ölçeklenebilirlik | |
| Modern uygulama gereksinimlerine uyum | |

Öğrenci avantajları şöyle açıklar: servisler birbirinden izole çalışır ve farklı ihtiyaçlar için farklı konteynerler oluşturulabilir; konteynerler fiziksel makinenin kaynaklarını düşük ve verimli kullanır; hızlı dağıtılır ve taşınır; ihtiyaca göre hızla yeni servis kaldırılabilir; günümüzün mikroservis yapısıyla uyumludur. Dezavantajlarda ise onlarca servisin yönetimi, yapılandırılması ve farklı makinelerde çalıştırılması karmaşık bir süreçtir; bunun için Kubernetes ya da Docker Swarm gibi orkestrasyon araçları gerekir. Güvenlik açısından yanlış oluşturulmuş bir imaj bütün makinelere dağılacağı için küçük de olsa bir risk vardır.

## 6. Neden bir monoliti yeniden yapılandıralım?

İkinci sunum mikroservislere yeniden yapılandırmayı (refactoring) ele alır. Üç alt başlık vardır: mikroservislere yeniden yapılandırmaya genel bakış, bir monolitin neden yeniden yapılandırıldığı ve monoliti boğma (strangling) kavramı.

### 6.1 Kitaptaki çıkış noktası

Öğrenci kitaptaki örnekten başlar: büyük bir şirketin monolitik uygulamasından sorumlu bir yönetici vardır. Mevcut sürümde mühendislik ekibi yeni özellikleri hızlı ve güvenilir şekilde sunamamaktadır. Çözüm olarak mikroservislere geçiş öne çıkar. Bölümde monolitik sistemden mikroservislere geçişin nasıl değerlendirileceği, geçiş stratejileri ve sorunların doğru tespiti tartışılır.

### 6.2 Avantajlar ve maliyet

Slayttaki maddeler şöyledir (ekrandan, 00:24:30):

- Mikroservis mimarisinin bakımı, test edilebilirliği ve dağıtımı daha kolaydır; bu da geliştirmeyi hızlandırır.
- Mikroservis yapısı daha ölçeklenebilirdir ve hata izolasyonunu iyileştirir; ayrıca teknoloji yığınını güncellemek çok daha kolaydır.
- Bir monoliti mikroservislere yeniden yapılandırmak büyük bir iştir ve yeni özellik geliştirmesinden kaynakları çeker.

Bu yüzden öğrenciye göre mikroservis mimarisine geçişin gerçekten önemli bir iş problemini çözmesi ve işletme tarafından desteklenmesi gerekir.

### 6.3 Monolitik cehennem

Monolitik cehennem (monolithic hell), monolitik mimariye sahip uygulamaların karşılaştığı zorlukları ve sorunları tanımlar. Bir monolitik cehennemdeysek büyük olasılıkla en az bir iş problemimiz vardır:

- Yavaş teslimat: monolitik yapılarda küçük bir değişiklik bile bütün uygulamanın yeniden dağıtılmasını gerektirir; bu da geliştiricilerin değişiklik yapma hızını düşürür.
- Hatalı yazılım sürümleri: monolitik yapıları test etmek zordur; test edilemeyen yazılım hatalarla dolu olur, bu da müşteri ve gelir kaybına yol açar.
- Verimsiz ölçekleme: bütün parçalar tek yerden yönetildiği için uygulamanın farklı bölümlerini ayrı ayrı büyütmek zorlaşır.

### 6.4 Önce sorunun kaynağını değerlendirmek

Öğrenci, bu sorunların gerçekten mimarinin sınırlarından mı kaynaklandığının önce değerlendirilmesi gerektiğini vurgular. Yavaş teslimat ve hatalı sürümler çoğu zaman kötü bir yazılım geliştirme sürecinin sonucudur. Örneğin manuel testlerden otomatik testlere geçmek geliştirme hızını ciddi şekilde artırabilir. Aynı şekilde ölçeklenebilirlik sorunları bazen mimariyi değiştirmeden çözülebilir. Sorunlar devam ederse mikroservis mimarisine geçiş düşünülmelidir.

## 7. Monoliti boğmak: Strangler deseni ve stratejileri

### 7.1 Monoliti boğma

Monoliti boğma (strangling the monolith), büyük bir uygulamayı parça parça, küçük ve bağımsız bölümlere, yani mikroservislere dönüştürme yöntemidir. Bu yöntemde yeni özellikler doğrudan mikroservis olarak geliştirilir; eski sistemin parçaları da ihtiyaç oldukça ve vakit buldukça tek tek mikroservislere dönüştürülür. Zamanla bütün sistem mikroservis yapısına geçer ve o büyük, karışık yapı küçülerek ortadan kalkar.

Öğrencinin saydığı avantajlar:

- Bütün sistemi bir kerede değiştirmek yerine parça parça geçiş yapılır; yönetim kolaylaşır.
- Sistem sürekli çalışır durumda kalır. Bir yandan değişiklik yapılırken işleyen bir sistem vardır; kesintiler en aza iner.
- Esneklik sağlar: projenin bir kısmı geliştirilirken başka bir kısmı dönüştürülmeye devam edilebilir.

Monolitik bir uygulamayı mikroservislere dönüştürmek, eski sistemi modernize etmek olarak da adlandırılır. Öğrenciye göre en iyi yaklaşım, bütün yapıyı bir kerede değiştirmeden adım adım ilerlemektir. Bunun yöntemi Strangler Fig desenidir: yeni mikroservisler eklenir ve eski sistemin işlevselliği yavaş yavaş bu servislere taşınır. Sistem sürekli çalışır durumda tutulduğu için geçiş daha güvenli ve kademelidir. Öğrenciye göre büyük firmalar genelde bu yolu tercih eder.

> **Ek bilgi (derste anlatılmadı):** “Strangler fig” adı, bir ağacın çevresinde büyüyüp zamanla onu saran ve sonunda yerini alan boğucu incir bitkisinden gelir. Deseni bu benzetmeyle büyük olasılıkla ilk adlandıran Martin Fowler’dır (2004). Benzetme, yeni servislerin monolitin çevresinde büyüyüp onu zamanla gereksiz kılmasını anlatır.

### 7.2 Strangler uygulamasının stratejileri

Öğrenci desenin uygulanmasında üç strateji sayar:

- Hızlı değer katma: kademeli geçişin yatırım getirisi erken sağlanmalıdır. Her yeni geçiş adımı ya da her yeni servis olumlu bir etki yaratmalıdır.
- Yaygın değişikliklerden kaçınma: büyük değişiklikler yerine küçük ve stratejik adımlarla geçiş yapılır.
- Altyapıya başta büyük yatırım yapmama: başlangıçta altyapı için büyük harcamalardan kaçınılır; mikroservis deneyimi kazanıldıkça altyapıyı genişletmek daha mantıklıdır.

### 7.3 Özet

Öğrencinin özetine göre monolitik uygulamaları servislere dönüştürmek karmaşık ama gerekli bir dönüşümdür. Strangler deseni kademeli bir yaklaşım önerir ve erken değer sunmaya olanak tanır. Yine de çözümün mevcut problemlerin doğru analizine dayandığından emin olmak gerekir; bazen daha basit çözümler yeterlidir. Projeyi bir anda baştan sona mikroservise geçirmek hata olasılığını çok artırdığı için daha küçük parçalarla ilerlemek önerilir. Sonunda zaten bütünüyle mikroservislere geçilmiş olur, ama süreçteki risklere en az ölçüde maruz kalınır.

## 8. Monolitik mimari ve geçişin gerekçeleri

Üçüncü sunum, bir önceki sunumu farklı ayrıntılarla genişleterek monolitik sistemlerin mikroservislere dönüştürülmesi için stratejileri anlatır.

### 8.1 Monolitik mimari

Monolitik mimari, bütün bileşenlerin tek bir kod tabanında bulunduğu ve birbirine sıkı bağlı olduğu yapıdır. Basit bir geliştirme süreci sunduğu ve başlangıçta hızlı sonuç verdiği için tercih edilir. Tek bir dağıtım birimi vardır, sıkı bağlı bileşenlerden oluşur ve merkezi bir yönetime sahiptir.

Avantajları:

- Basit ve kolay geliştirme: bütün bileşenler tek kod tabanında olduğu için geliştiriciler bütün sistemi tek yerden kontrol edip test edebilir.
- Performans: bütün bileşenler aynı sunucuda ya da merkezi bir sistemde çalıştığı için iletişim gecikmeleri en aza iner.
- Dağıtım ve bakım kolaylığı: tek kod tabanı ve tek uygulama dosyasıyla çalışmak dağıtımı ve bakımı kolaylaştırır.
- Yeni projelere uygunluk: küçük ekiplerin geliştirdiği uygulamalarda zaman ve kaynak tasarrufu sağlar.

### 8.2 Dezavantajlar

Başlangıçta basit ve hızlı geliştirme sunsa da uygulama büyüdükçe sorunlar ortaya çıkar:

- Karmaşıklık ve bağımlılık: bir bileşende yapılan değişiklik diğer bileşenlerde zincirleme etkiler yaratır; bu da test sürecini karmaşıklaştırır.
- İş birliği: kod tabanı büyüdükçe ekipler arasında iş birliği zorlaşır, birden çok geliştiricinin aynı anda çalışması verimsizleşir.
- Ölçeklenebilirlik: bütün sistemin gereksiz yere ölçeklenmesi maliyeti artırır ve süreçleri karmaşıklaştırır.
- Bakım zorluğu: kod tabanı büyüdükçe bakım süreçleri giderek karmaşıklaşır.

Bu dezavantajlar yüzünden mikroservislere geçiş için stratejiler geliştirilmiştir. Öğrenciye göre geçiş doğrudan olmaz; önce mevcut yapıyı koruyan bir altyapı sağlanmalı, bileşenler çeşitli stratejilerle eklenmeli ya da dönüştürülmelidir.

## 9. Üç geçiş stratejisi

### 9.1 Strangler Fig modeli

Slayttaki anlatım şöyledir (ekrandan, 00:36:15):

- Yaklaşım: mevcut monolitik yapı tamamen yeniden yazılmadan dönüştürülür.
- Adımlar: (1) yeni işlevler mikroservisler olarak eklenir; (2) eski bileşenler mikroservislere kademeli olarak taşınır; (3) zaman içinde bütün fonksiyonlar mikroservislere aktarılır.
- Avantaj: işlevsellik kesintiye uğramaz ve dönüşüm aşamalı olarak gerçekleştirilir.

Slayttaki şekilde zaman ekseni boyunca “Strangler Application” kutusunun içindeki mikroservis sayısı adım adım artar, alttaki monolitik blok ise küçülür ve sonunda tamamen kaybolur. Öğrencinin anlatımıyla monolitik servisler ile mikroservisler bir süre birlikte yaşar (coexistence). Yeni eklenenler mikroservislerle karşılıklı entegre edilir. Mikroservisler işi tamamen

üstlendikten sonra monolit boğulur ve tamamen kaldırılır. Dönüşüm sürecinde iş ihtiyaçlarından doğan yeni talepler de mikroservis mimarisi üzerinde ele alınmaya başlanır. Öğrenci "strangler" sözcüğünün "boğan" anlamına geldiğini, desenin adının buradan geldiğini belirtir.

### 9.2 Yeni özellikleri servis olarak uygulamak

Yeni özellik mantığının monolitin kod tabanına değil, mikroservis olarak eklenmesi sistemin karmaşıklığının artmasını önler ve geliştirme hızını artırır. Slayttaki maddeler (ekrandan, 00:37:30):

- Faydalar: monolitin büyümesini engeller; yeni özellikler hızlı geliştirilir ve dağıtılır; mikroservislerin değerini hızla gösterir.
- Entegrasyon: API Gateway bütün istekleri yönlendirir; glue code, monolit ile yeni servisler arasındaki veri paylaşımını ve işlevsel bağlantıları sağlar.

Slayttaki kitap şeklinde API Gateway eski özellik isteklerini ("Old features") monolitin inbound adapter'ına, yeni özellik isteklerini ("New features") yeni servisin inbound adapter'ına yönlendirir. Yeni özelliği uygulayan servis DelayedDeliveryService, Order ve Notification aggregate'lerini ve kendi veritabanını içerir. Monolit ile servis arasındaki "Integration glue", servisin outbound adapter'ından monolitin inbound adapter'ına giden çağrılardan ve monolitin event publisher adapter'ından servisin event subscriber adapter'ına giden event'lerden oluşur.

Öğrenci integration glue'yu şöyle tanımlar: hizmeti monolitle entegre eder; hizmetin monolitin sahip olduğu verilere erişmesini ve monolit tarafında uygulanan işlevselliği çağırmasını sağlar. Öğrenciye göre iletişimi entegre etmek zordur. Bu yüzden iletişimi koparmadan, iki entegrasyon aracını (API Gateway ve glue code) kullanarak yavaş yavaş entegre etmek doğru yoldur.

### 9.3 Sunum katmanını arka uçtan ayırmak

Slayttaki anlatım (ekrandan, 00:38:45) bir uygulamanın üç katmanını verir:

- Sunum katmanı: kullanıcı arayüzü.
- İş mantığı katmanı: iş kurallarını uygular.
- Veri erişim katmanı: veritabanı ve mesajlaşma sistemlerine erişim sağlar.

Tipik bir kurumsal uygulamada sunum mantığı ile iş ve veri erişim mantığı arasında temiz bir ayrım vardır. İş katmanının kaba taneli (coarse-grained) bir API'si bulunur; bu API, monoliti iki küçük uygulamaya bölmek için doğal bir dikiş yeridir. Slayttaki şekilde sunum mantığını ve arka uç iş mantığını birlikte içeren bir monolit ("Browser → HTML pages → Web application → Business logic → Database adapter → MySQL") ikiye bölünür (Split):

- Bağımsız dağıtılabilen, daha küçük bir sunum mantığı monoliti: Browser → HTML pages → Web app → REST client.
- Bağımsız dağıtılabilen, daha küçük bir arka uç monoliti: REST API → Business logic → Database adapter → MySQL. Bu REST API, ileride yazılacak servislerin de çağırabileceği bir API'dir.

Bölmeden sonra sunum uygulaması iş mantığı uygulamasına uzaktan çağrı yapar. Öğrenciye göre amaç, katmanları ayırırken aradaki iletişimi tamamen kaybetmemektir. Slayttaki faydalar: bağımsız geliştirme ve dağıtım; A/B testi ve kullanıcı deneyiminde hızlı iterasyon.

### 9.4 İşlevsel yetenekleri servislere taşımak

Slayttaki maddeler (ekrandan, 00:40:00):

- İşlevlerin mikroservislere taşınması monolitin küçülmesini sağlar.
- İş mantığı daha iyi ölçeklenir.
- Dikey dilimler: API uç noktaları, iş mantığı ve veritabanı erişim mantığı bağımsız olarak taşınır.

Slayttaki kitap şeklinde monolitteki Order Service'in bir kısmı ("Code to extract into a service") ayrılır. Sağdaki yeni servis kendi Order Service'ini, Courier ve Plan aggregate'lerini ve

kendi veritabanını içerir. Monolitte Order Service ve Order aggregate'i kalır. API Gateway istekleri iki tarafa yönlendirir; aradaki "Integration glue" iki yönlü inbound/outbound adapter çiftlerinden oluşur ("Glue code integrating service with monolith").

## 10. Servis çıkarmanın zorlukları: domain modeli ve veritabanı

### 10.1 Domain modelini parçalamak

Slayttaki tanıma göre (ekrandan, 00:41:20) domain modeli, uygulamanın iş kurallarını ve veritabanı yapısını temsil eden merkezi bileşendir. Monolitten mikroservise geçerken bu modelin parçalanması ve mikroservislere uyarlanması gerekir. Süreç iki nedenle karmaşıktır:

- Nesne referansları: monolitik yapılarda bileşenler arası referanslar sıkıdır. Mikroservis yapısına geçişte bu referanslar birincil anahtarlarla değiştirilmelidir.
- Bağımsız domain modelleri: her mikroservis kendi domain modeline sahip olduğundan, bu modellerin bağımsız ve tutarlı şekilde tanımlanması gerekir.

Bir servisi çıkarmak için onun domain modelinin monolitin domain modelinden çıkarılması gerekir. Asıl sorun, servis sınırlarını aşacak nesne referanslarını ortadan kaldırmaktır: monolitte kalan sınıflar servise taşınan sınıflara referans verebilir ya da tersi olabilir. Slayttaki şekil şunu gösterir:

```
Önce:   FTGO monolith: [Order] ---> [Restaurant]

Servis çıkarılınca:
        Delivery Service: [Order] --?--> FTGO monolith: [Restaurant]
        (servis sınırını aşan nesne referansı)

Çözüm:  Delivery Service: [Order | restaurantId]     FTGO monolith: [Restaurant]
        (referans birincil anahtarla değiştirilir)
```

Öğrencinin anlatımıyla Order sınıfı çıkarıldığında Restaurant'a olan referans kaybolur; çünkü Restaurant monolitte kalmıştır. Bu sorunu çözmek için DDD aggregate'lerindeki kural uygulanır: aggregate'ler birbirine nesne referansıyla değil, birincil anahtarla referans verir. Order ve Restaurant birer aggregate olarak düşünülür. Order sınıfındaki Restaurant referansı, Restaurant'ın birincil anahtarını saklayan bir `restaurantId` alanıyla değiştirilir. Böylece ilişki kaybolmadan servis sınırı aşılmış olur.

### 10.2 Veritabanını yeniden yapılandırmak

Domain modelini bölmek kodu değiştirmekten fazlasını içerir; çünkü domain modelindeki birçok sınıf kalıcıdır, alanları bir veritabanı şemasıyla eşleştirilmiştir. Bir servisi monolitten çıkarınca verisi de taşınır: tablolar monolitin veritabanından servisin veritabanına taşınmalıdır. Bir varlık bölündüğünde ilgili tablo da bölünür ve yeni tablo servise eklenir.

Slayttaki maddeler (ekrandan, 00:44:25):

- Tabloların bölünmesi: monolitik sistemdeki tablolar mikroservis tabanlı yapılara bölünmelidir; örneğin müşteri ve sipariş tablosu ilgili mikroservislere taşınabilir.
- Replikasyon ve proxy kullanımı: geçiş sürecinde veritabanı erişimi için replikasyon ve salt okunur proxy'ler kullanılabilir; bu, veri tutarlılığını koruyarak geçişi kolaylaştırır.

Slayttaki şekil şöyledir:

```
Önce:
  FTGO monolith: Order entity (consumerId, scheduledPickupTime, scheduledDeliveryTime, ...)
                 ORDER tablosu (ORDER_ID, RESTAURANT_ID, ..., SCHEDULED_PICKUP_TIME, SCHEDULED

Servis çıkarılıp teslimat sütunları ORDER tablosundan yeni DELIVERY tablosuna taşınınca:
  FTGO monolith: Order entity; teslimatla ilgili alanlar salt okunur
                 ORDER tablosu (aynı sütunlar)
  Delivery Service: Delivery entity (orderId, scheduledPickupTime, scheduledDeliveryTime, ...)
                    DELIVERY tablosu (ORDER_ID, ..., SCHEDULED_PICKUP_TIME, SCHEDULED_DELIVERY
  Veri Delivery Service'ten FTGO monolitine replike edilir.
```

Öğrencinin açıklamasıyla, veriyi kopyalayarak Order varlığının yapısını monolitte korumak, hemen yapılması gereken iş miktarını önemli ölçüde azaltır. Teslimatla ilgili Order alanlarını ya da ORDER tablosu sütunlarını kullanan kod zamanla diğer servise taşınabilir; bazı durumlarda monolitte hiç değişiklik yapmak gerekmeyebilir. Bu kod daha sonra bir servise çıkarılırsa, o servis veriye diğer servis üzerinden erişebilir.

### 10.3 Bağımlılık yönetimi ve geçişin faydaları

Monolitin en büyük dezavantajlarından biri bağımlılıktı; mikroservislere geçişte de bağımlılıkların yönetimi önemlidir. Öğrenci iki ilke verir:

- Bağımsız mikroservisler: geçiş sırasında her mikroservis bağımsız hâle getirilir, diğer servisleri en az düzeyde etkilemesi hedeflenir.
- API tabanlı iletişim: mikroservisler arası iletişim açıkça tanımlanmış API'ler üzerinden yapılmalıdır.

Geçişin faydaları: mikroservisler bağımsız geliştirilip dağıtıldığı için hızlı geliştirme ve dağıtım; ihtiyaç duyulan bileşenler ayrı ölçeklendiği için daha iyi ölçeklenebilirlik ve daha düşük maliyet; hata izolasyonu sayesinde artan güvenilirlik; her mikroservis için en uygun teknolojinin seçilebildiği esneklik. Zorluk ise teknik ve organizasyonel değişiklik gereksinimidir: stratejileri doğru uygulamak, taşıma sırasında veri kaybını önlemek ve doğru iletişimi sağlamak gerekir. Öğrencinin sonucuna göre mikroservis mimarisi büyüyen ve karmaşık projeler için bir gerekliliktir; doğru strateji ve planlamayla dönüşüm en yüksek faydayı sağlar.

## 11. Servis ile monolitin iletişimi: integration glue

Dördüncü sunum, monolitten ayrılan bir servisin monolitle nasıl iş birliği yapacağını ele alır.

### 11.1 İki mimari kanun

Öğrenci sunuma, sorunları yorumlarken faydalı bulduğu iki yazılım mimarisi kanunuyla başlar:

- Yazılım mimarisindeki her şey bir ödünleşimdir (trade-off). Bir şeyin ödünleşim olduğunu fark etmediysen, henüz fark etmemişsin demektir.
- Neden, nasıldan daha önemlidir.

> **Ek bilgi (derste anlatılmadı):** Bu iki kanun büyük olasılıkla Mark Richards ve Neal Ford'un *Fundamentals of Software Architecture* kitabından alınmıştır. Kitapta birinci kanun "Everything in software architecture is a trade-off", ikinci kanun "Why is more important than how" diye geçer.

### 11.2 Servis ayırmanın gerekçesi ve integration glue

Önceki sunumlarda anlatıldığı gibi yazılım monolit ve mikroservislerden oluşan bir yapıda bir arada bulunabilir. Monolitten servis ayırma, yazılıma yeni bir özellik eklenmek istendiğinde ya da monolitin performansı düştüğünde, monolitin sorumluluklarını azaltıp mikroservislere aktarmak için yapılır; bu da performansı artırabilir. Sunum, servis ile monolit arasındaki iletişim ve veri tutarlılığının nasıl korunacağı üzerinde durur.

Ayrılan iki bileşenin haberleşmesi gerekir; kitap bu iletişimi integration glue diye adlandırır. Slayttaki kitap şeklinin altyazısına göre (ekrandan, 00:50:00) monolitten mikroservislere geçerken servisler ve monolit çoğu zaman birbirinin verisine erişmek zorundadır. Bu etkileşimi, API'leri uygulayan adapter'lardan oluşan integration glue sağlar; bazı API'ler mesajlaşma tabanlı, bazıları RPC tabanlıdır.

> **Ders sonrası not:** Öğrenci "integration glue" teriminin internette aranınca çıkmadığını, bu yüzden kabul görmemiş bir terim olduğunu yorumladı. Terim büyük olasılıkla Richardson'ın kitabına özgü bir adlandırmadır; kavram ise genel olarak entegrasyon katmanı ya da adapter kodu diye bilinir.

### 11.3 Integration glue tasarım adımları

Slayttaki adımlar (ekrandan, 00:50:00):

1. Mikroservisin hangi API'leri sunacağına karar verilir.
2. API kullanıcılarının prosedürün iç mantığını bilmesine gerek yoktur; bu yüzden iç mantık gizlenir (encapsulation). Öğrenci bunu OOP'deki kapsüllemeye benzetir.
3. Entegrasyon stili ve IPC mekanizması seçilir.

### 11.4 Kitaptaki örnek ve alternatif tasarım

Slayttaki kitap şekli (Figure 13.9) şöyledir (ekrandan, 00:51:20):

```
Delayed Delivery Service
  CustomerContactInfoRepository --> REST client
        --- GET /customers/{customerId} --->
                                      REST API --> FTGO monolith --> Monolith database
```

Bu örnekte monolitten ayrılmış Delayed Delivery Service, müşteri iletişim bilgisini almak için monolitin REST API'sini çağırır; böylece monolitin veritabanından okur. CustomerContactInfoRepository arayüzünü uygulayan adapter bu API'yi çağırır. Slayttaki değerlendirme:

- REST olması geliştirme açısından basitlik sağlar, ama performansı yüksek değildir.
- Monoliti meşgul edebilir.
- Senkron IPC olması iki bileşeni de yavaşlatabilir.
- Ne sıklıkla sorgulama yapılacak? Çok sıksa alternatif tasarımlar düşünülebilir.

Öğrenciye göre burada akla gelmesi gereken desen CQRS'tir; çünkü çok fazla okuma yapılırsa monolit meşgul olur. Kitaptaki alternatif tasarımda bir mesaj kuyruğu ve CQRS deseni kullanılır: monolit event'leri mesaj kanalına yazar, mikroservis bunlarla kendi veritabanını monolitin veritabanıyla güncel tutar. Slayta göre mesaj kanalı sistemi karmaşıklaştırır, ama monolitin yükü azalır. Öğrenci bu tasarımın getirdiği ek sorumluluğu da belirtir: monolit veritabanıyla servis veritabanının senkronize tutulması gerekir.

### 11.5 İzolasyon eksikliği ve diğer dezavantajlar

Öğrenciye göre bu tasarımda izolasyon eksikliği (lack of isolation) ortaya çıkar. SQL veritabanları ACID ilkelerini sağlar, ama saga'lar yalnız ACD ilkelerini sağlar. Bir yazma işlemi yapıldığında, bu değişikliğin servisteki görünüme kopyalanmasına kadar geçen sürede bir boşluk oluşur; bu aralıkta izolasyon eksikliği yaşanır.

> **Ek bilgi (derste anlatılmadı):** ACID; atomiklik (atomicity), tutarlılık (consistency), yalıtım (isolation) ve kalıcılık (durability) ilkelerinin baş harfleridir. Saga'larda her adım ayrı bir yerel işlem olarak hemen commit edildiği için, bir saga'nın ara sonuçları diğer saga'lara görünür; yani "I" (izolasyon) sağlanmaz ve geriye ACD kalır. Bu yüzden saga'larda izolasyon eksikliği countermeasure denen tasarım teknikleriyle ele alınır.

Öğrencinin saydığı başka dezavantajlar da vardır: eski monolit eski programlama dilleri ve kütüphanelerle yazılmış olabilir ve ekipte bunları bilen kimse kalmamış olabilir.

## 12. Anti-corruption Layer deseni

### 12.1 İki domain modeli sorunu

Öğrenci, yazılım mühendisliği derslerinden bilinen legacy sistem yöntemlerine değinir ve bunları ayrıntılı anlatmaz. Burada iki yol vardır: ya monolitin domain modeli baştan yazılır, ya da var olan model değiştirilir. Bir alternatif de ikisini aynı anda kullanmaktır: yeni servisler yeni domain modelini, monolit eski domain modelini kullanır. Monolitin domain modelini değiştirmek çok tehlikeli olduğu için eski model monolitte sürdürülür. Aradaki geçiş Anti-corruption Layer (ACL) deseniyle sağlanır.

### 12.2 Desenin yapısı

Slayttaki kitap şekli (Figure 13.11) şöyledir (ekrandan, 00:53:50):

```
Delayed Delivery Service                          FTGO monolith
  CustomerContactInfoRepository (API)
     |  (servisin ortak dili - ubiquitous language of service)
     v
  [Translation layer]   <- Anti-corruption layer
  [Monolith layer]         (monolitin ortak dili - ubiquitous language of monolith)
  [REST client] ---- GET /user/{userId} ----> REST API --> FTGO monolith
```

Şeklin açıklamasına göre monolitin API'sini çağıran servis adapter'ı, servisin domain modeli ile monolitin domain modeli arasında çeviri yapmak zorundadır. Slayttaki maddeler:

- Eski monolit domain modelinin yeni domain modelini kirletmesinin önüne geçmemizi sağlar.
- OOP'deki Adapter deseni gibidir.

Öğrencinin vurguladığı önemli bir ayrıntı, ACL katmanının yeni servise konmuş olmasıdır, monolitte bulunmaz. Öğrenciye göre bu katmanı mikroservise koymak daha az risklidir ve orada daha bilinen teknolojiler kullanılır. Kitapta da bu katmanın eski domain modelinin yeni domain modelini kirletmesini önlediği belirtilir. ACL farklı bileşenlerde olabilir ve aradaki iletişimin yönü de farklı olabilir. Öğrenci, sunumun başında söz ettiği ödünleşimlerin (eski teknoloji gibi) bu tasarımda da geçerli olduğunu hatırlatır.

## 13. Saga'larla veri tutarlılığı ve kimlik doğrulama

### 13.1 Saga terimlerinin hatırlatılması

Veri tutarlılığına geçmeden önce öğrenci kitaptaki temel saga terimlerini hatırlatır (ekrandan, 00:55:10). Slayttaki tanımlar:

- Saga: asenkron mesajlaşmayla koordine edilen yerel işlemler dizisi.
- Compensating transaction: bir yerel işlemin yaptığı güncellemeleri geri alan işlem.
- Countermeasure: saga'lar arasındaki izolasyon eksikliğini ele almak için kullanılan tasarım tekniği.
- Semantic lock: bir saga tarafından güncellenmekte olan kayda bir bayrak koyan countermeasure.
- Compensatable transaction: saga'da kendisinden sonra gelen işlemlerden biri başarısız olabileceği için telafi işlemine ihtiyaç duyan işlem.
- Pivot transaction: saga'nın devam/dur (go/no-go) noktası. Başarılı olursa saga sonuna kadar çalışır.
- Retriable transaction: pivot işleminden sonra gelen ve başarılı olması garanti olan işlem.

Öğrenci özellikle countermeasure, semantic lock ve pivot transaction terimlerini vurgular. Countermeasure'lar saga'lar arasındaki izolasyon eksikliğini yönetmek ve takip etmek için kullanılır; semantic lock, veritabanlarındaki okuma/yazma kilitlerine benzer. Pivot transaction saga'nın dönüm noktasıdır; buraya gelindikten sonra geri dönüş olmaz ve compensating transaction yapılmaz.

### 13.2 Kitchen Service örneği

Öğrenciye göre kitapta, mikroservislerin monolitten çıkarılma sırasının compensating transaction yazma ihtiyacını ortadan kaldırabildiği gösterilir; doğru çıkarma sırası izolasyon eksikliğini yönetmeyi kolaylaştırır. Slayttaki kitap metni şöyledir (ekrandan, 00:56:20):

Kitchen Service monolitten çıkarılırken Order entity'si bölünür ve Kitchen Service'te bir Ticket entity'si oluşturulur. Bu, monolitin `createOrder()` dahil birçok komutunu etkiler. Monolit `createOrder()` komutunu tek bir ACID işlemi olarak şu adımlarla uygular:

1. Sipariş ayrıntılarını doğrula.
2. Tüketicinin sipariş verebildiğini doğrula.
3. Tüketicinin kredi kartını yetkilendir.

4. Bir Order oluştur.

Bu ACID işleminin yerine şu adımlardan oluşan bir saga konmalıdır:

```
1. Monolitte:
   - Order, APPROVAL_PENDING durumunda oluşturulur.
   - Tüketicinin sipariş verebildiği doğrulanır.
2. Kitchen Service'te:
   - Sipariş ayrıntıları doğrulanır.
   - Ticket, CREATE_PENDING durumunda oluşturulur.
3. Monolitte:
   - Tüketicinin kredi kartı yetkilendirilir.
   - Order'ın durumu APPROVED olur.
4. Kitchen Service'te:
   - Ticket'ın durumu AWAITING_ACCEPTANCE olur.
```

Öğrenci slaytta adımların yanına monolit ile servis arasında gidip gelen oklar ve başarısız olabilecek adımları gösteren çarpılar çizmiştir. Öğrencinin anlatımıyla akış monolitten başlar; APPROVAL_PENDING ve CREATE_PENDING durumları semantic lock (countermeasure) örnekleridir. Önce monolitin veritabanına bu alan yazılır, sonra Kitchen Service'in veritabanına bir pending durumu yazılır ve akış böyle devam eder. Sonraki örnekler compensating transaction'ın nasıl önlenebileceğini gösterir: sonraki adım başarısız olmuyorsa compensating transaction yapmaya da gerek kalmaz.

### 13.3 Kimlik doğrulama ve yetkilendirme

Kitabın bu bölümde son olarak ele aldığı konu yetkilendirme ve kimlik doğrulamadır. Monolitin kullandığı yetkilendirme ve kimlik doğrulama yöntemleri mikroservisinkinden farklı olabilir. Kitapta ikisinin aynı anda nasıl yönetileceği API Gateway üzerinden gösterilir: monolit ve mikroservisin farklı yöntemleri API Gateway'de toplanır ve yetkilendirme ile kimlik doğrulama yönetimi monolite hiç dokunmadan gateway ile gerçekleştirilir.

## 14. Bileşen testleri: stub, mock ve Gherkin

Son sunum, önceki haftadan kalan bileşen (component) testlerinin geliştirilmesi konusudur.

### 14.1 Bileşen testi nedir?

Bileşen testi, bir yazılım bileşeninin izole olarak test edilmesini sağlayan yöntemdir. Amaç, bileşenin dış API'sini kullanarak işlevselliğini ve doğru çalıştığını doğrulamaktır. Test piramidinde entegrasyon testleri ile uçtan uca (end-to-end) testler arasında yer alır ve onları tamamlayıcı olarak görülür. Bu testler, bileşenin bağımlılıklarını simüle eden stub ya da mock nesneleri kullanılarak yapılır.

### 14.2 Stub

Slayttaki tanım (ekrandan, 01:00:00): stub, test edilen kodun bağımlı olduğu bir bileşenin yerine kullanılan, sabit (önceden tanımlı) bir davranış sergileyen test çiftidir. Stub yalnızca test edilen kodun ihtiyaç duyduğu sabit bir çıktıyı döndürmekle ilgilenir.

- Önceden tanımlı cevaplar döner: belirli girdilere karşılık sabit bir cevap döner, dinamik bir davranış göstermez.
- Dış bağımlılıkları izole eder: test edilen kod gerçek bileşenle değil, stub ile çalışır.
- Çağrıları izlemez: kendisine yapılan çağrıları kaydetmez ya da analiz etmez.

Stub, test edilen kodun belirli bir fonksiyonu çalıştırabilmesi için gereken bir bileşeni simüle etmek istendiğinde kullanılır. Örneğin bir API'den veri çeken bir fonksiyonu test etmek için gerçek API'ye ihtiyaç duymadan stub kullanılabilir.

### 14.3 Mock

Mock, bir bileşeni simüle ederken aynı zamanda kendisine yapılan çağrıları kaydeden ve bu çağrıların analizini sağlayan test çiftidir. Dış bileşenle yapılan etkileşimlerin doğruluğunu kontrol etmek için kullanılır. Örneğin bir e-posta gönderim fonksiyonunun doğru parametrelerle çağrıldığını test etmek için mock kullanılabilir.

Öğrencinin özetiyle: stub gerekli test verisini sağlamak için kullanılır; mock ise test edilen kodun dış bileşenlerle doğru şekilde etkileşim kurduğunu kontrol etmek için kullanılır.

### 14.4 Kabul testleri ve Gherkin

Kabul testleri, bir yazılım ürününün belirlenen gereksinimleri ve kullanıcı ihtiyaçlarını karşıladığını doğrulamak için yapılan son aşama testleridir. Genellikle müşteri ya da son kullanıcı tarafından onaylanmış gereksinimlere dayanır; test senaryoları ve kullanım durumları kullanıcıya göre belirlenir.

Bu testler Gherkin ile yazılır. Gherkin, davranış güdümlü geliştirme (behavior-driven development, BDD) çerçevesinde test senaryolarını tanımlamak için kullanılan bir alana özgü dildir (DSL). Kullanıcı hikayelerini ve test senaryolarını hem teknik hem de teknik olmayan ekiplerin anlayabileceği, insan tarafından okunabilir bir biçimde yazmayı sağlar; böylece teknik olmayan ekipler yazılımın nasıl davranması gerektiğini anlayabilir. Senaryo üç anahtar sözcükle kurulur:

- Given: başlangıç durumu, testin hangi koşullarda başlayacağı.
- When: test sırasında gerçekleşen eylem.
- Then: bu girdilerle eylem gerçekleştikten sonra beklenen sonuç (çıktı).

Cucumber, Gherkin ile yazılmış senaryoları koda bağlayan araçtır.

### 14.5 Cucumber ile Gherkin örneği

Slayttaki kitap örneği şöyledir (ekrandan, 01:03:40). Sol tarafta Gherkin özelliği:

```
Feature: Place Order

  As a consumer of the Order Service
  I should be able to place an order

  Scenario: Order authorized
    Given a valid consumer
    Given using a valid credit card
    Given the restaurant is accepting orders
    When I place an order for Chicken Vindaloo at Ajanta
    Then the order should be APPROVED
    And an OrderAuthorized event should be published

  Scenario: Order rejected due to expired credit card
    Given a valid consumer
    Given using an expired credit card
    Given the restaurant is accepting orders
    When I place an order for Chicken Vindaloo at Ajanta
    Then the order should be REJECTED
    And an OrderRejected event should be published
```

Sağ tarafta bu adımları koda bağlayan sınıf:

```
public class StepDefinitions ... {
  ...
  @Given("A valid consumer")
  public void useConsumer() { ... }

  @Given("using a(.?) (.*) credit card")
```

```
  public void useCreditCard(String ignore, String creditCard) { ... }

  @When("I place an order for Chicken Vindaloo at Ajanta")
  public void placeOrder() { ... }

  @Then("the order should be (.*)")
  public void theOrderShouldBe(String desiredOrderState) { ... }

  @And("an (.*) event should be published")
  public void verifyEventPublished(String expectedEventClass) { ... }
}
```

Öğrencinin anlatımıyla FTGO (Food to Go) uygulaması için senaryonun başında koşullar belirlenir: geçerli bir tüketici, geçerli bir kredi kartı ve siparişleri kabul eden bir restoran. Sonra sipariş verilir; koşullar sağlanıyorsa sipariş onaylanır ve siparişin kabul edildiğini bildiren event yayınlanır. Her adım, yandaki sınıfta bir yönteme karşılık gelir. Düzenli ifadedeki `(.*)` gruplar adımdaki değişken kısmı (kartın türü, beklenen sipariş durumu, event sınıfı) yönteme parametre olarak aktarır.

## 15. Bileşen testlerinin tasarımı, yazılması ve çalıştırılması

### 15.1 Tasarım: in-process ve out-of-process

Bir bileşen testinde Gherkin senaryosu çalıştırılmadan önce servisin (örnekte Order Service) çalıştırılması ve bağımlılıklarının ayarlanması gerekir. Order Service izole test edileceği için bir veritabanı ve mesajlaşma altyapısı kurulmalıdır. Gerçekçiliği hız ve basitlikle dengeleyen iki seçenek vardır:

- In-process bileşen testleri: servis, bağımlılıkları bellek içi stub ve mock nesnelerle değiştirilerek testle aynı JVM içinde çalıştırılır. HSQL gibi bellek içi bağımlılıklar kullanılabilir. Yazması daha basit ve daha hızlıdır. Ancak dağıtılabilir servisi, yani üretime çıkacak hâlini test etmez.
- Out-of-process bileşen testleri: servis üretime hazır biçimiyle ayrı bir süreçte çalışır; bunun yaygın yolu Docker konteyner imajı kullanmaktır. Gerçek altyapı servisleri kullanılır, uygulama servisi bağımlılıkları ise stub ile değiştirilir. FTGO Order Service testinde MySQL ve Apache Kafka gerçek altyapıdır; Consumer Service ve Accounting Service gibi bağımlılıklar stub olarak yapılandırılır. Bu yaklaşım daha gerçekçidir ve test kapsamını artırır, ama daha yavaş, daha kırılgan ve daha karmaşıktır.

### 15.2 FTGO Order Service bileşen testlerinin yazılması

Slayttaki anlatım (ekrandan, 01:07:15):

```
@RunWith(Cucumber.class)
@CucumberOptions(features = "src/component-test/resources/features")
public class OrderServiceComponentTest {
}
```

- Sınıf, Gherkin özellik dosyalarının nerede bulunacağını belirten bir `@CucumberOptions` açıklamasına sahiptir.
- JUnit'e Cucumber test çalıştırıcısını kullanmasını söyleyen `@RunWith(Cucumber.class)` ile açıklanmıştır. Tipik bir JUnit test sınıfının aksine hiçbir test yöntemi yoktur. Bunun yerine Gherkin özelliklerini okuyarak testleri tanımlar ve bunları yürütülebilir hâle getirmek için `OrderServiceComponentTestStepDefinitions` sınıfını kullanır.
- `OrderServiceComponentTestStepDefinitions` bir test sınıfı olmamasına rağmen, Spring Testing çerçevesinin bir parçası olan `@ContextConfiguration` ile açıklanmıştır.

Öğrencinin gösterdiği genel mimari şekilde, Gherkin ile verilen senaryo Cucumber tarafından çevrilir ve OrderServiceComponentTest tarafından okunur; Order Service, Docker ile Apache Kafka ve MySQL gibi altyapı hizmetleriyle birlikte çalıştırılır ve diğer servisler simüle edilir.

### 15.3 Step definitions sınıfı

Slayttaki kod ve açıklamalar şöyledir (ekrandan, 01:09:20):

```
@ContextConfiguration(classes =
    OrderServiceComponentTestStepDefinitions.TestConfiguration.class)
public class OrderServiceComponentTestStepDefinitions {

  ...

  @Autowired
  protected SagaParticipantStubManager sagaParticipantStubManager;

  @Given("using a(.?) (.*) credit card")
  public void useCreditCard(String ignore, String creditCard) {
    if (creditCard.equals("valid"))
      sagaParticipantStubManager                          // başarı cevabı gönder
          .forChannel("accountingService")
          .when(AuthorizeCommand.class).replyWithSuccess();
    else if (creditCard.equals("invalid"))
      sagaParticipantStubManager                          // başarısızlık cevabı gönder
          .forChannel("accountingService")
          .when(AuthorizeCommand.class).replyWithFailure();
    else
      fail("Don't know what to do with this credit card");
  }
```

- `OrderServiceComponentTestStepDefinitions` sınıfı testlerin kalbidir; Order Service'in bileşen testlerindeki her adımın anlamını tanımlar. Gösterilen `useCreditCard()` yöntemi, "Given using ... credit card" adımının anlamını tanımlar.
- Bu yöntem, saga katılımcıları için stub'ları yapılandıran bir test yardımcı sınıfı olan `SagaParticipantStubManager`'ı kullanır. Accounting Service stub'ını, belirtilen kredi kartına bağlı olarak (valid ya da invalid) başarı ya da başarısızlık mesajıyla yanıt verecek şekilde yapılandırır.
- `placeOrder()` yöntemi "When I place an order for Chicken Vindaloo at Ajanta" adımını tanımlar. Siparişi oluşturmak için Order Service'in REST API'sini çağırır ve yanıtı sonraki bir adımda doğrulamak için kaydeder.
- `baseUrl()` yardımcı yöntemi Order Service'in URL'sini döndürür.

### 15.4 Testlerin çalıştırılması

Slayttaki anlatım (ekrandan, 01:10:00):

- Testler Order Service'i ve bağımlılıklarını çalıştırmak için Docker'ı kullanır. Docker Compose, bir dizi konteyner tanımlanıp bunların bir birim olarak başlatılıp durdurulabildiği son derece kullanışlı bir araçtır. FTGO uygulamasının kök dizininde bütün hizmetler için konteynerleri ve altyapı hizmetlerini tanımlayan bir `docker-compose` dosyası vardır. Testleri yürütmeden önce konteynerleri çalıştırmak ve testler bitince durdurmak için Gradle Docker Compose eklentisi kullanılır.
- Bu testler nispeten yavaş olduğu için `./gradlew test` komutunun bir parçası olarak çalıştırılmaz. Test kodu ayrı bir `src/component-test/java` dizinine konur ve `./gradlew componentTest` ile çalıştırılır.
- Gradle yapılandırması iki şey yapar. Birincisi, Gradle Docker Compose eklentisini bileşen testlerinden önce çalışacak ve Order Service'i, bağımlı olduğu altyapı hizmetleriyle birlikte başlatacak şekilde yapılandırır. İkincisi, Docker imajının gerektirdiği JAR dosyasının önce derlenmesi için `componentTest` görevini `assembly`'ye bağımlı yapar.

Bu komutla bileşen testleri dört adımda çalışır:

1. Order Service derlenir (build).
2. Servis ve altyapı servisleri başlatılır.
3. Testler çalıştırılır.
4. Çalışan servisler durdurulur.

## Dersin Toparlaması

Bu derste beş öğrenci sunumuyla mikroservis kitabının dağıtım, yeniden yapılandırma ve test bölümleri işlendi. İlk sunumda servislerin bağımlılıklarıyla birlikte konteynerlere paketlendiği Service as a Container deseni anlatıldı. Konteynerlerin, her biri kendi işletim sistemini çalıştıran sanal makinelere göre hafif olduğu, Docker'ın bileşenleri (Engine, Image, Registry, Dockerfile), build ve runtime aşamaları ile Docker Compose ve Kubernetes gibi orkestrasyon araçlarının rolü gösterildi. Sonraki iki sunumda monoliti mikroservislere dönüştürmenin gerekçeleri ve yolları işlendi. Önce sorunun gerçekten mimariden kaynaklanıp kaynaklanmadığı analiz edilmeli, geçiş ise Strangler deseniyle kademeli yapılmalıdır. Bunun üç yolu yeni özellikleri servis olarak yazmak, sunum katmanını arka uçtan ayırmak ve işlevsel yetenekleri servislere çıkarmaktır. Servis çıkarırken servis sınırını aşan nesne referansları birincil anahtarla değiştirilir, tablolar bölünür ve veri replike edilir.

Dördüncü sunumda servis ile monolitin iş birliği ele alındı. Integration glue'nun tasarımı, REST ile mesajlaşma tabanlı CQRS arasındaki ödünleşim, iki domain modeli arasında çeviri yapan Anti-corruption Layer, saga terimleri ve servis çıkarma sırasının compensating transaction ihtiyacına etkisi anlatıldı. Son sunumda bileşen testleri, stub ve mock farkı, Gherkin ve Cucumber ile senaryo yazımı, in-process ve out-of-process testler ile FTGO Order Service örneğinin Gradle ve Docker Compose ile çalıştırılması gösterildi. Hoca, bu dersle mikroservis mimarisinin ana hatlarıyla, değişik yönleri ve desenleriyle incelenmesinin tamamlandığını söyledi.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.